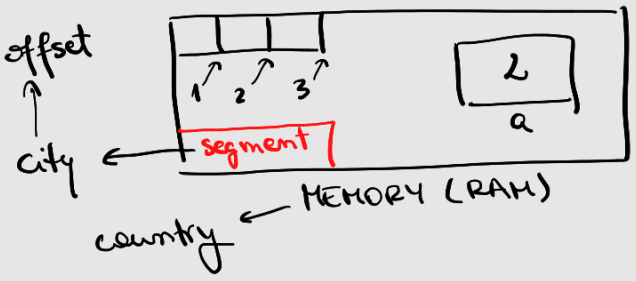
resb → for asm, not CPU.

How can we retrieve data from memory?



data segment ....
a db 2;
↙
name of variable
(address)

code segment...
⋮
mov AL, [a] ← value
mov EAX, a ← address

ASSEMBLY	C
[a]	*a

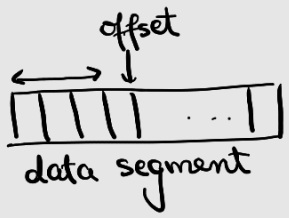
→ dereferentiating in order to see the value.

1) Address of memory location: number of consecutive bytes starting from the beginning of the memory (RAM)

2) Segment: a logical section of the program's memory.
↳ Characteristics: - basic address (where does the segment start, beginning of the segment)
- limit (size, 32 bits for their limits)

8086-based processors:
- a block of memory of discrete size, physical segment
  16 bit CPU => 64K
  32 bit CPU => 4 Gb ?
- variable size block of memory

the CPU tells us how much to allocate.



3) offset = the address of a location relative to the beginning of a segment.

When is an offset valid?
- ↳ if it is a numerical value on 32 bits.
- ↳ doesn't exceed the segments' limit.

Address specification (in Hexa)

$s_3 s_2 s_1 s_0 : \theta_7 \theta_6 \theta_5 \theta_4 \theta_3 \theta_2 \theta_1 \theta_0$

- $s_3 s_2 s_1 s_0$: segment selector (16 bits) → provided by OS (we don't need to specify it)
- $\theta_7 \theta_6 \theta_5 \theta_4 \theta_3 \theta_2 \theta_1 \theta_0$: offset (32 bits) → 8 hexa letters

→ **FAR ADDRESS** ↓ because we specify the segment selector

$b_7 b_6 b_5 b_4 \dots b_1 b_0$ = base of the segment

$l_7 l_6 l_5 l_4 \dots l_1 l_0$ = limit (size) of the segment

$$\theta_7 \theta_6 \theta_5 \theta_4 \theta_3 \theta_2 \theta_1 \theta_0 \leq l_7 l_6 l_5 l_4 l_3 l_2 l_1 l_0 \quad (*)$$

↓

offset ≤ limit.

The actual address will be computed in the following way:

$$\underbrace{a_7 a_6 a_5 \dots a_1 a_0}_{32 \text{ bits}} = \underbrace{b_7 b_6 \dots b_1 b_0}_{32 \text{ bits (base)}} + \underbrace{\theta_7 \theta_6 \dots \theta_1 \theta_0}_{32 \text{ bits}}$$

Why on earth $32 = 32 + 32$??

Ex: 8 : 1000h . Compute the LINEAR ADDRESS of this.

8 → segment selector; 1000h → offset

So, the CPU does this before storing / retrieving data.

↳ How? CPU checks if offset ≤ limit (*)

Base of segment: 2000h

Limit of segment: 4000h

Checks (*): 1000h $\overset{?}{\leq}$ 4000h

This kind of addressing = SEGMENTATION ADDRESS MODEL.

Is this offset valid?

↳ when the segment { starts from 0 ; has the max size 4Gb } => VALID OFFSET.

FLAT MEMORY MODEL:

$a_7 a_6 \dots a_1 a_0 = 0000\ 0000 + a_7 a_6 a_5 \dots a_1 a_0$.

PAGING MODEL:

OS divides the virtual memory into pages.

(OS —translate→ physical memory)

8086 execution models:

- real memory (on 16 bits CPU)
- protected mode (on 16/32 bits CPU) → uses PAGING / SEGMENTATION models.
- virtual mode
- long mode (64/32 bit CPU) → PAGING / ~~SEGMENTATION~~

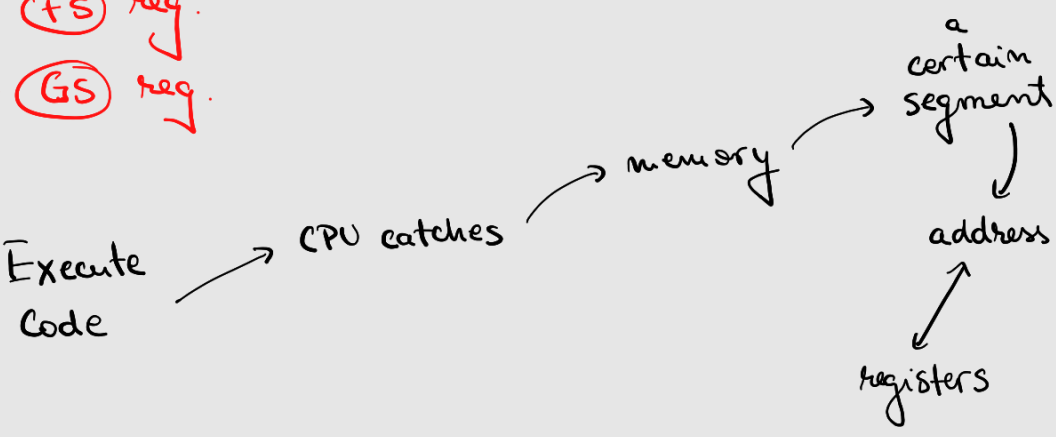
x86 architecture:

- CODE SEGMENT (a location in memory)

↳ instructions: → (CS) reg. → check the value given by OS, but we cannot modify it.

- DATA SEGMENT
  ↳ data → (DS) reg.
- STACK SEGMENT
  ↳ local computations etc. → (SS) reg.
- EXTRA SEGMENT
  ↳ suplimentary DATA SEGMENT → (ES) reg.

+ (FS) reg.
+ (GS) reg.



EIP → stores the pointer of the CURRENT instruction.
→ cannot be modified
→ can be seen
→ offset in (CS) = the address of a location relative to the beginning of (CS)

NOTION	REPRESENTATION	DESCRIPTION
FAR ADDRESS ↴ address specification	selector : offset	defines the segment and offset
selector	16 bits	identifies an available segment in OS.
offset ↴ NEAR ADDRESS	32 bits	defines only the offset component.
LINEAR ADDRESS = SEGMENTATION ADDRESS MODEL	32 bits	beginning of segment + offset.
	≥ 32 bits	final result of

PHYSICAL EFFECTIVE ADDRESS

SEGMENTATION

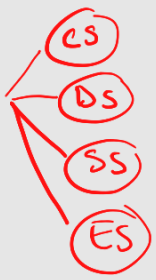
PAGING

computed by BIU.

FAR VS. NEAR ADDRESS:

NEAR address: give only the offset

↳ always inside one of the 4 active segments. (depends on the instructions used)



FAR address: complete address specification.

① $s_3 s_2 s_1 s_0 : o_7 o_6 o_5 \dots o_1 o_0$

② segment reg : offset

③ FAR variable : type QWORD , little endian.

Computing the offset of an operand:

1) mov EAX, 17 (register mode)

2) mov EAX, 17 (immediate mode)

? ↳ I don't need to specify the offset for immediate mode.

3) operand located in memory (memory addressing mode)

↳ I need to retrieve the op. from memory, so I need the offset.

↳ In this case,

$$\text{offset\_address} = [\text{base}] + [\text{index} * \text{scale}] + [\text{ct}]$$

base: EAX, EBX, ECX, EDX, EBP, ESI, EDI and ESP.

index: 1 ——— 4 ———, no ESP (because ESP is already an index)

scale: multiply the index with 1 → byte
2 → word
4 → dword

8 → qword

ct: the value of a numerical constant (byte / word / dword)

"[ ]" → because it can be / not be in the formula.

Modes of addressing memory:

1. DIRECT addressing (only the [ct] is present)
2. BASE addressing (one of the [base] reg. is present)
3. SCALE_INDEX addressing ([index] reg. present in the offset specif.)

mov EAX, [08A...] (=) mov EAX, [X]

mov [EAX], DWORD 1734 (specify the size → 4 bytes = 32 bits (EAX)).

" DWORD

↓

go in memory, copy 1734 and store it on 32 bits.

mov EAX, 1734

↳ should be on 32 bits.